

## ラマの障害物 (Obstacles for a Llama)

ラマはアンデス高原内を移動したい。ラマはこの高原の地図を持っており、地図は  $N \times M$  のマス目の形をしている。地図の行には上から順に  $0$  から  $N - 1$  までの番号が付けられており、列には左から順に  $0$  から  $M - 1$  までの番号が付けられている。行  $i$  列  $j$  ( $0 \leq i < N$ ,  $0 \leq j < M$ ) にある地図のマスを  $(i, j)$  と表記する。

ラマはこの高原の気候について学び、地図の各行について、その行にあるすべてのマスが同じ **温度** を持っていること、および地図の各列について、その列にあるすべてのマスが同じ **湿度** を持っていることを発見した。ラマはあなたに長さがそれぞれ  $N$  と  $M$  である  $2$  つの整数列  $T$  と  $H$  を与えた。 $T[i]$  ( $0 \leq i < N$ ) は行  $i$  にあるマスの温度を表し、 $H[j]$  ( $0 \leq j < M$ ) は列  $j$  にあるマスの湿度を表す。

ラマはこの高原の植生についても学び、マス  $(i, j)$  にはそのマスの温度がそのマスの湿度よりも高いとき、形式的には  $T[i] > H[j]$  であるとき、またそのときに限り **植物がない** ことに気づいた。

ラマは **正当なパス** のみによってこの地域内を移動することができる。正当なパスとは以下の条件を満たす、相異なるマスからなる列である：

- 列の中で隣り合うマス同士は、ある辺を共通して持つ。
- 列に含まれる全てのマスには植物がない。

あなたの課題は  $Q$  個の質問に答えることである。それぞれの質問では  $4$  つの整数  $L, R, S, D$  が与えられる。あなたは以下を満たすような正当なパスが存在するか判定する必要がある：

- パスはマス  $(0, S)$  から始まり、マス  $(0, D)$  で終わる。
- パスに含まれる全てのマスは列  $L$  から列  $R$  までの間 (列  $L, R$  を含む) に位置する。

マス  $(0, S)$  とマス  $(0, D)$  には共に植物がないことが保証される。

## 実装の詳細

まず、あなたは以下の関数を実装する必要がある：

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- $T$ : それぞれの行の温度を示す長さ  $N$  の数列。
- $H$ : それぞれの列の湿度を示す長さ  $M$  の数列。
- この関数は各テストケースにおいてちょうど一度だけ、`can_reach` が呼び出される前に呼び出される。

次に、あなたは以下の関数を実装する必要がある：

```
bool can_reach(int L, int R, int S, int D)
```

- $L, R, S, D$ : 質問を表す4つの整数.
- この関数は各テストケースにおいて  $Q$  回呼び出される.

この関数は全てのマスが列  $L$  から列  $R$  までの間に位置するようなマス  $(0, S)$  からマス  $(0, D)$  までの正当なパスが存在するとき、またそのときに限り `true` を返す必要がある.

## 制約

- $1 \leq N \leq 200\,000$ .
- $1 \leq M \leq 200\,000$ .
- $1 \leq Q \leq 200\,000$ .
- $0 \leq T[i] \leq 10^9$  ( $0 \leq i < N$ ).
- $0 \leq H[j] \leq 10^9$  ( $0 \leq j < M$ ).
- $0 \leq L \leq R < M$ .
- $L \leq S \leq R$ .
- $L \leq D \leq R$ .
- マス  $(0, S)$  とマス  $(0, D)$  には共に植物がない.

## 小課題

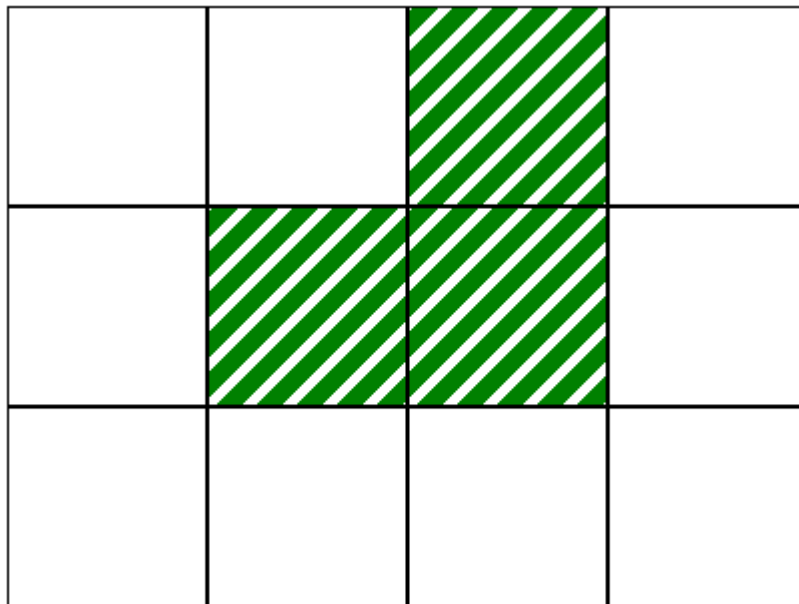
| 小課題 | 得点 | 追加の制約                                                                           |
|-----|----|---------------------------------------------------------------------------------|
| 1   | 10 | それぞれの質問において $L = 0$ , $R = M - 1$ .<br>$N = 1$ .                                |
| 2   | 14 | それぞれの質問において $L = 0$ , $R = M - 1$ .<br>$T[i - 1] \leq T[i]$ ( $1 \leq i < N$ ). |
| 3   | 13 | それぞれの質問において $L = 0$ , $R = M - 1$ .<br>$N = 3$ , $T = [2, 1, 3]$ .              |
| 4   | 21 | それぞれの質問において $L = 0$ , $R = M - 1$ .<br>$Q \leq 10$ .                            |
| 5   | 25 | それぞれの質問において $L = 0$ , $R = M - 1$ .                                             |
| 6   | 17 | 追加の制約はない.                                                                       |

## 例

次の呼び出しを考える.

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

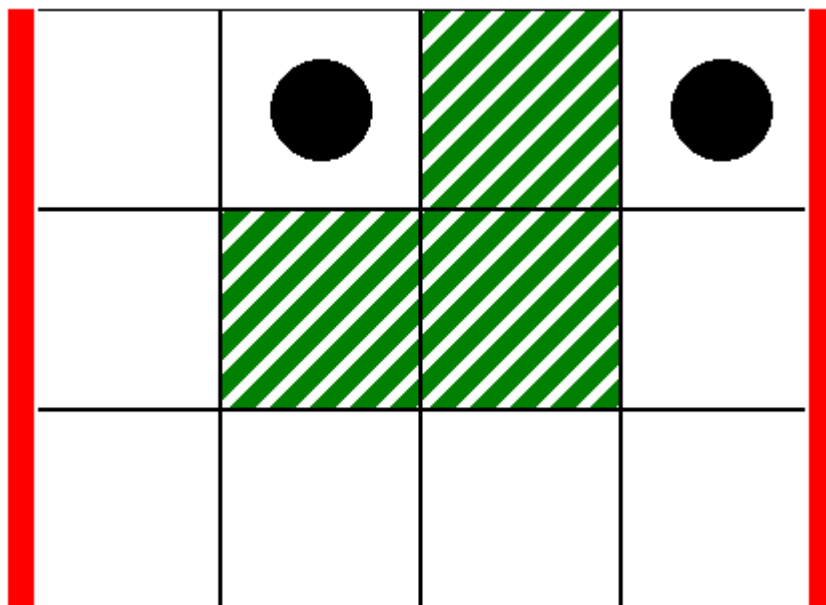
この呼び出しに対応する地図は次の図のようになる。白いマスには植物がない：



1 番目の質問として、次の呼び出しを考える。

```
can_reach(0, 3, 1, 3)
```

この質問は次の図に示す状況に対応する。縦に引かれた太い線が  $L = 0$  から  $R = 3$  までの列の範囲を示し、黒い円は最初マスと最後のマスを示す：



この場合、ラマはマス (0,1) からマス (0,3) まで以下の正当なパスを通して移動することができる：

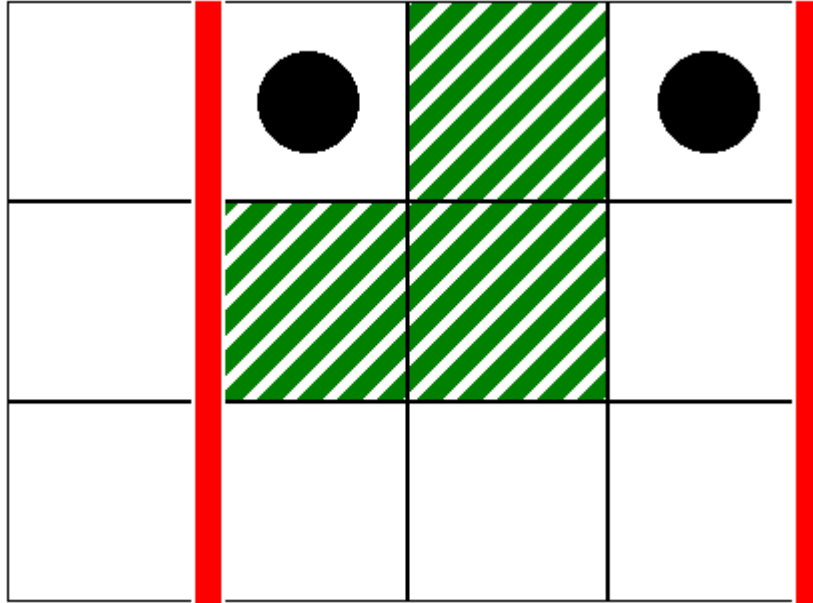
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

したがって、この関数は `true` を返す必要がある。

2 番目の質問として、次の呼び出しを考える。

```
can_reach(1, 3, 1, 3)
```

この質問は次の図に示す状況に対応する。



この場合、全てのマスが列 1 から列 3 までの間に位置するようなマス  $(0, 1)$  からマス  $(0, 3)$  までの正当なパスが存在しない。したがって、この関数は `false` を返す必要がある。

## 採点プログラムのサンプル

入力形式：

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

ここで、 $L[k], R[k], S[k], D[k]$  ( $0 \leq k < Q$ ) は `can_reach` のそれぞれの呼び出しの引数を示す。

出力形式：

```
A[0]  
A[1]  
...  
A[Q-1]
```

ここで、 $A[k]$  ( $0 \leq k < Q$ ) は `can_reach(L[k], R[k], S[k], D[k])` が `true` を返したとき 1 であり、そうでないとき 0 である。